

Operating System Principles

Module – 2

Lecture 4 - Threads

Thread

- A thread in an operating system is a **single sequential flow of execution of tasks within a process.**
- A thread is a basic unit of CPU utilization; it comprises a **thread ID, a program counter (PC), a register set, and a stack.**
- Lightweight process - shares resources and is faster compared to processes

Thread - Characteristics

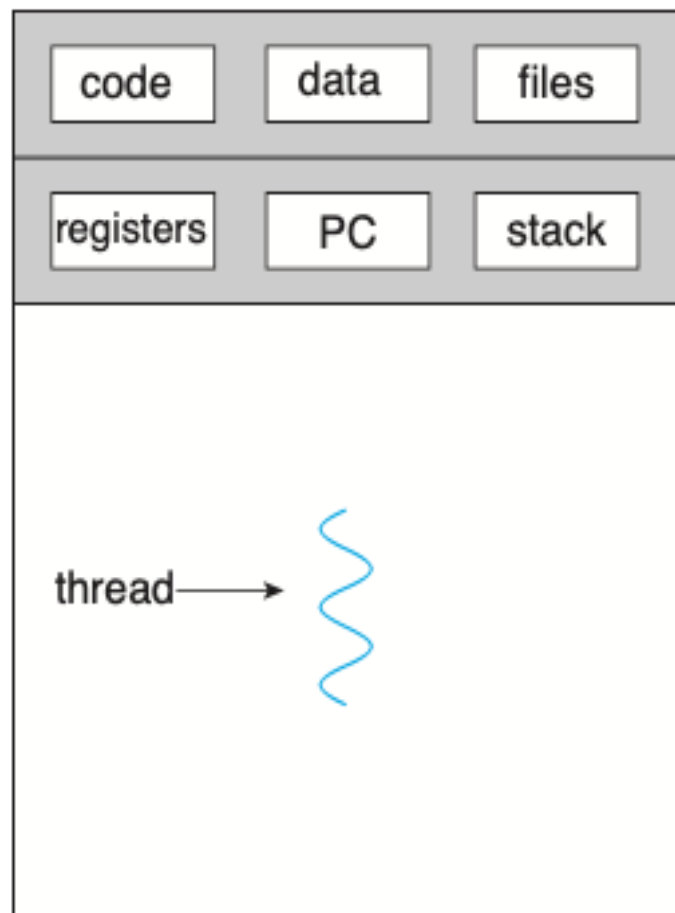
Lightweight: Threads are lighter than processes and require fewer resources.

Shared resources: Threads share the same code, data, and files.

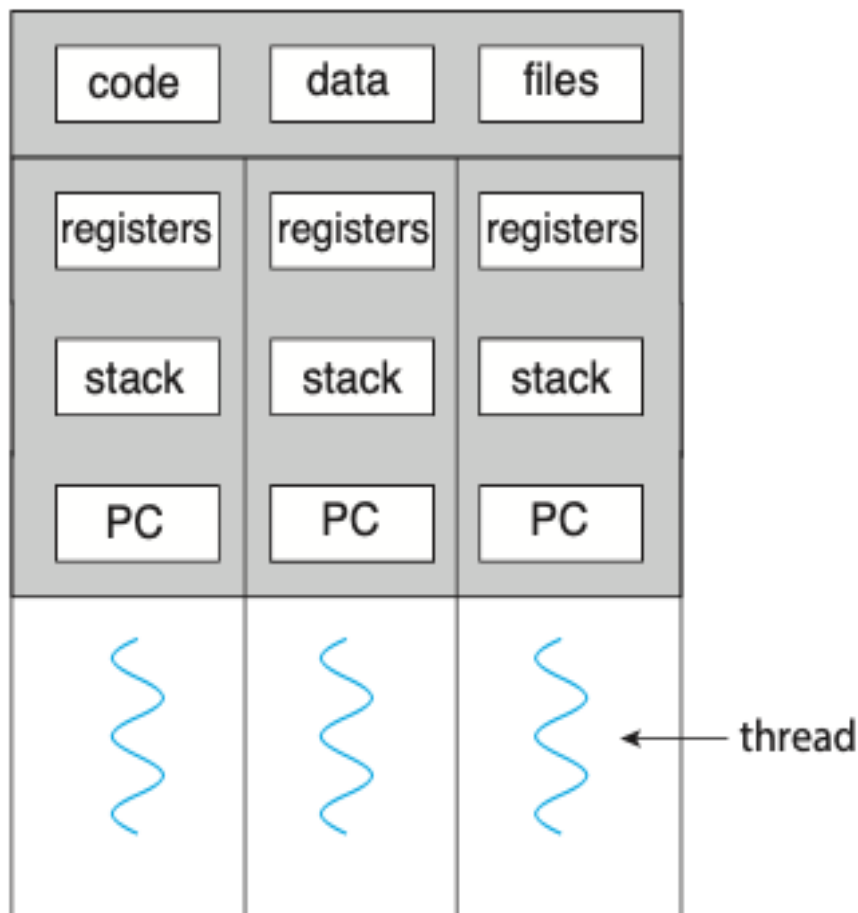
Fast context switching: Context switching between threads is faster than between processes.

Concurrent execution: Threads can run concurrently, improving system performance and responsiveness.

Single-threaded process vs Multithreaded process



single-threaded process



multithreaded process

Processes with Threads – Single threaded Advantages

- A single-threaded process is a process that contains only one thread, executing sequentially and having its own memory space.

Processes with Threads – Single threaded Characteristics

- Only one thread runs at a time
- No shared memory space
- No need for synchronization mechanisms
- Easier to implement and debug

Processes with Threads – Single threaded Advantages

- Simpler to implement and debug
- Less overhead in terms of memory and resources
- Easier to understand and maintain

Multithreads - Introduction

- A multithreaded process is a process that contains multiple threads, each executing concurrently and sharing the same memory space.
- Each thread has its own
 - Program counter,
 - Stack, and local variables,
 - But they share the same code, data, and files.

Multithreaded Characteristics

- Multiple threads run concurrently.
- Threads share the same memory space.
- Each thread has its own program counter, stack, and local variables.
- Threads can communicate with each other using synchronization mechanisms.

Multithreading

- Most software applications that run on modern computers and mobile devices are multithreaded
- An application typically is implemented as a separate process with several threads of control.
- For example: A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking.
- Web server – client example

Multithreading

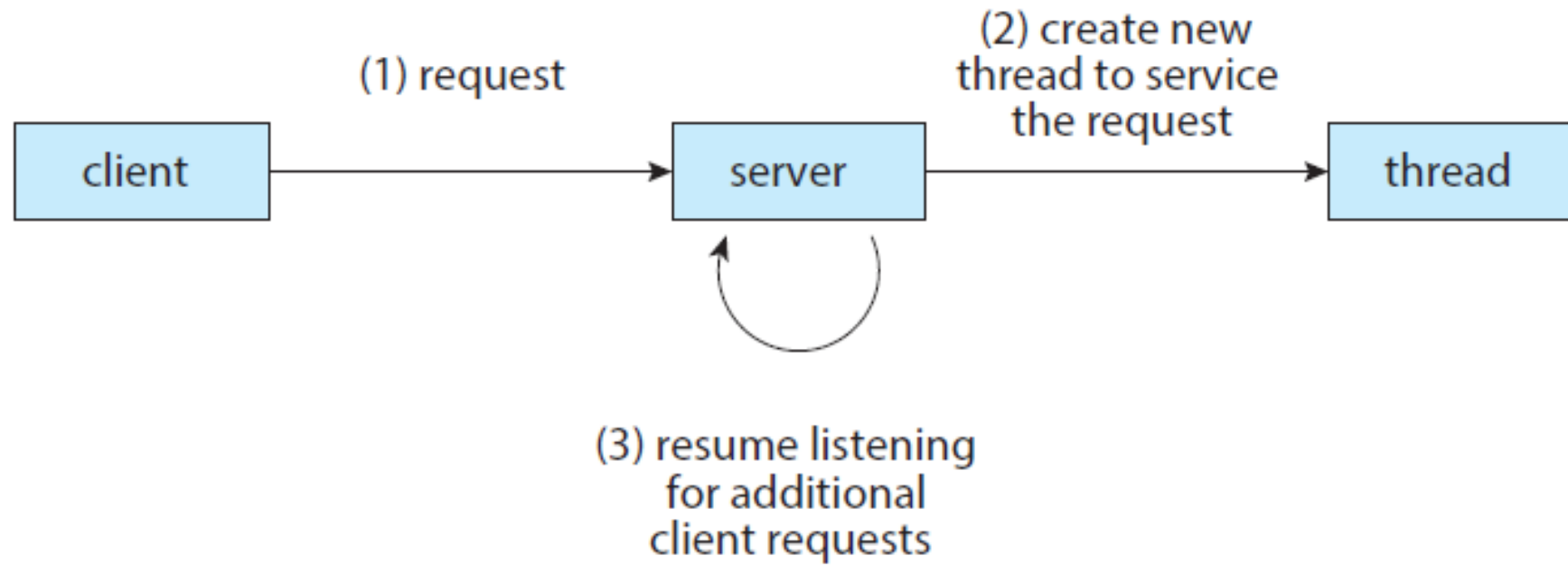
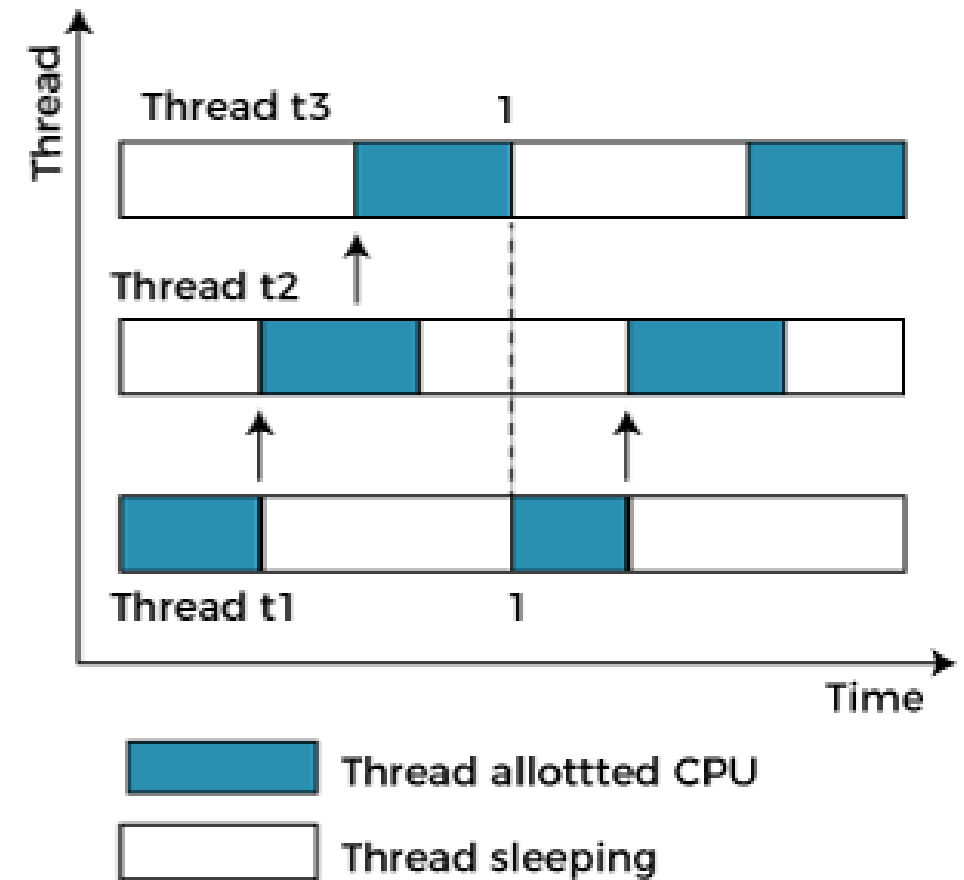
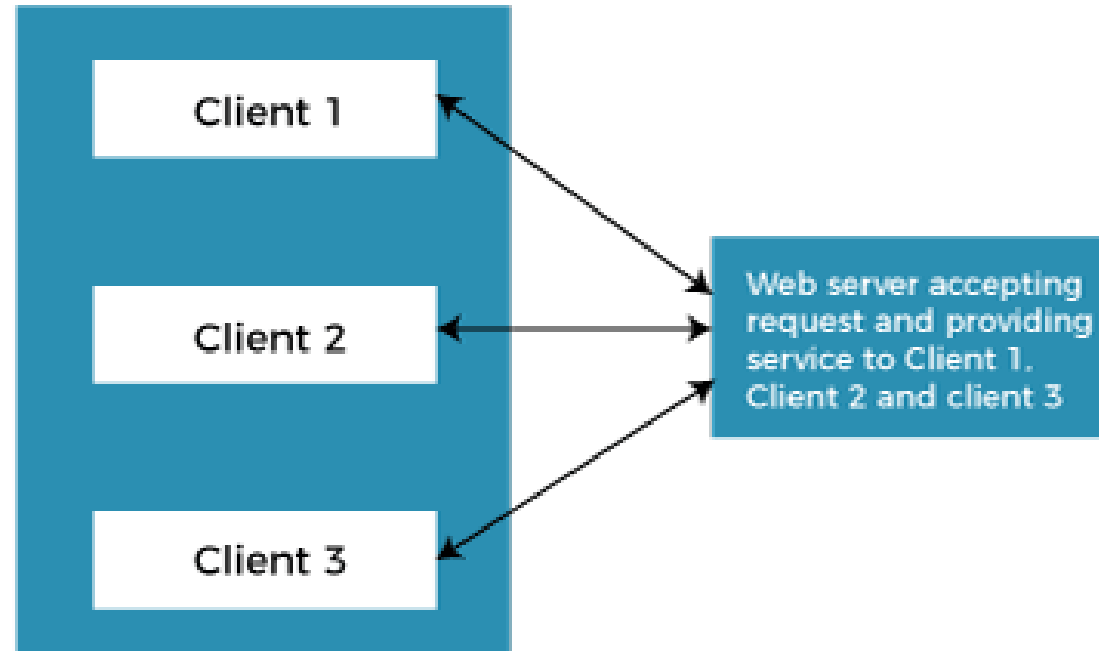


Figure: Multithreaded server architecture.

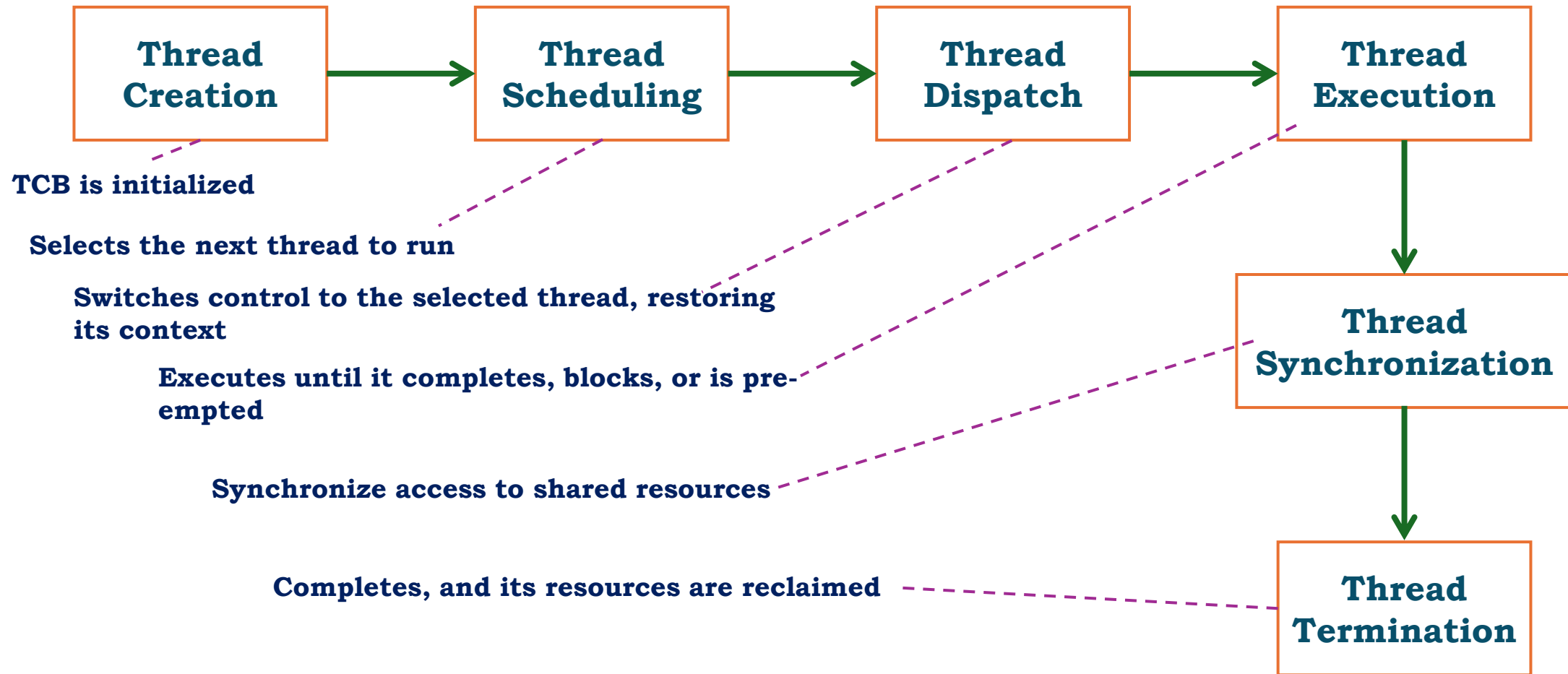
Multithreading



Multithreading

- Most operating system kernels are also typically multithreaded.
- Each thread performs a specific task, such as managing devices, memory management, or interrupt handling.
- The command **ps -ef** can be used to display the kernel threads on a running Linux system
- This command will show the kernel thread **kthreadd** (with pid = 2), which serves as the parent of all other kernel threads

Processes with Threads – Multithreaded Workflow



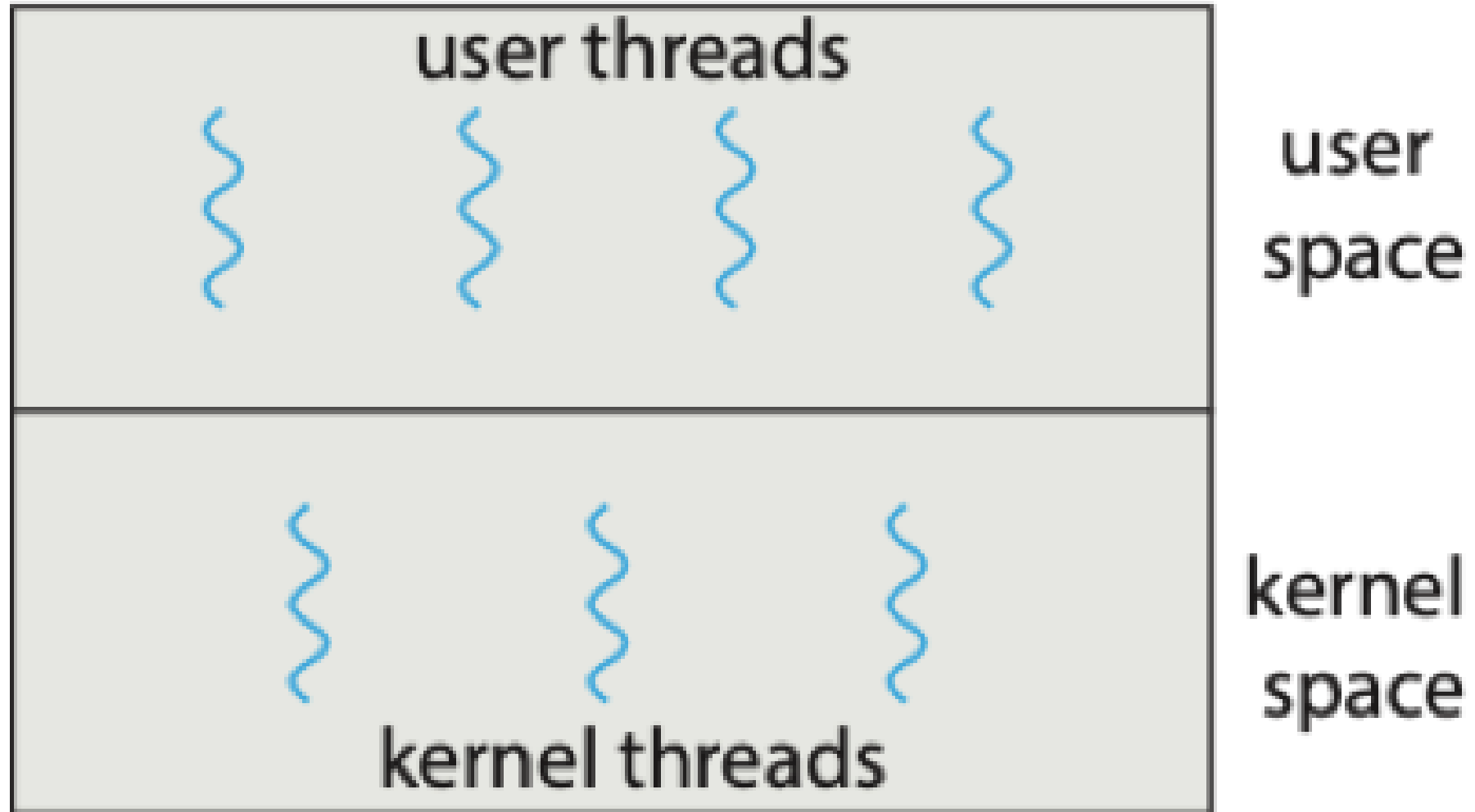
Multithreaded Advantages

- Improved responsiveness
- Increased throughput
- Better resource utilization
- Easier to implement concurrent programming

Multithreading Models

- A *user thread* is an entity used by programmers to handle multiple flows of controls within a program. The API for handling user threads is provided by the *threads library*.
- A kernel thread is the schedulable entity, which means that the system scheduler handles kernel threads.
- A kernel thread runs within a process, but can be referenced by any other thread in the system.
- User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system.

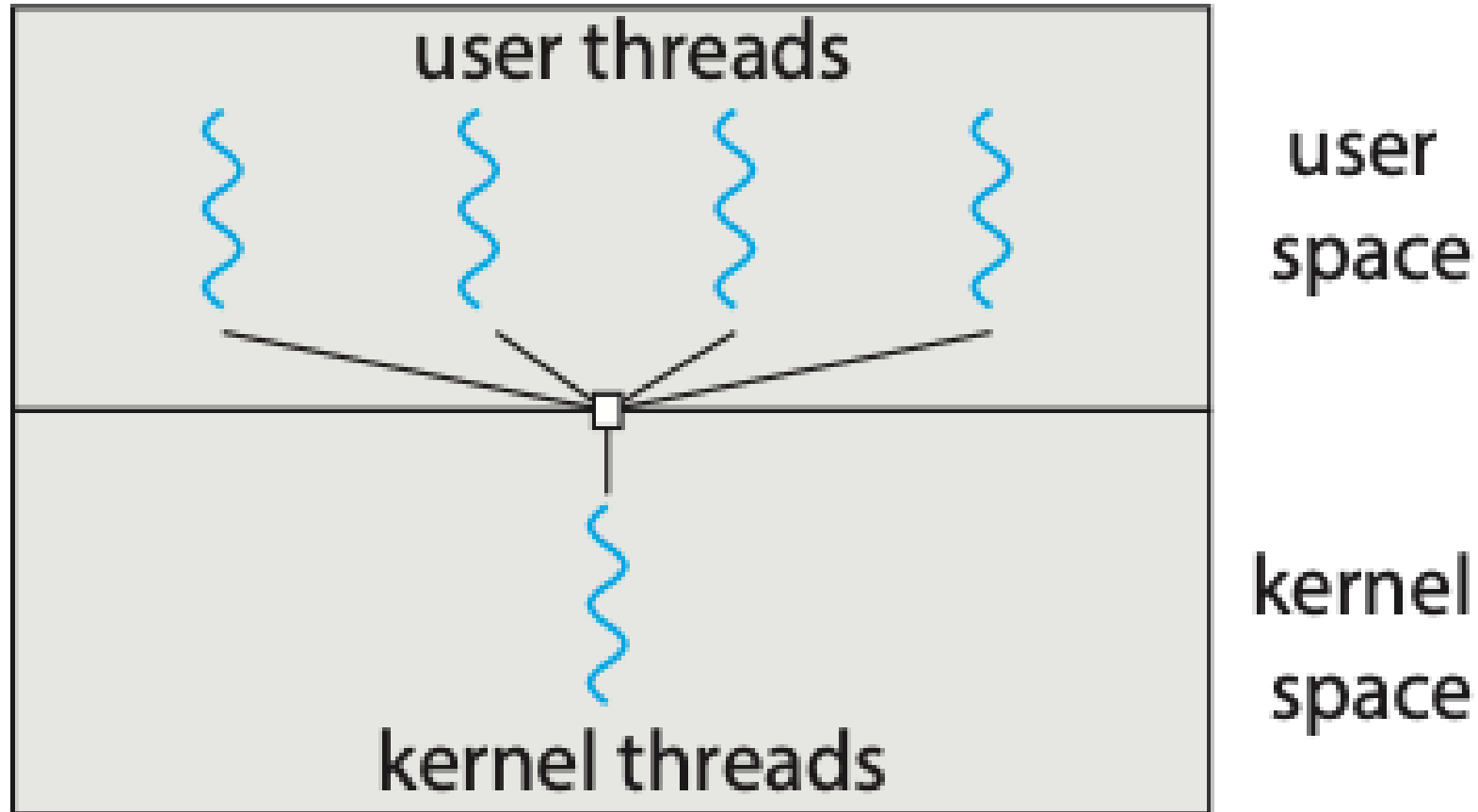
Multithreading Models



Multithreading Models

- **Many-to-One Model:** Multiple user threads map to a single kernel thread.
- **One-to-One Model:** Each user thread maps to a single kernel thread.
- **Many-to-Many Model:** Multiple user threads map to multiple kernel threads.
- **Two-Level Model:** A combination of many-to-one and one-to-one models.

Multithreading Models – Many to One Thread



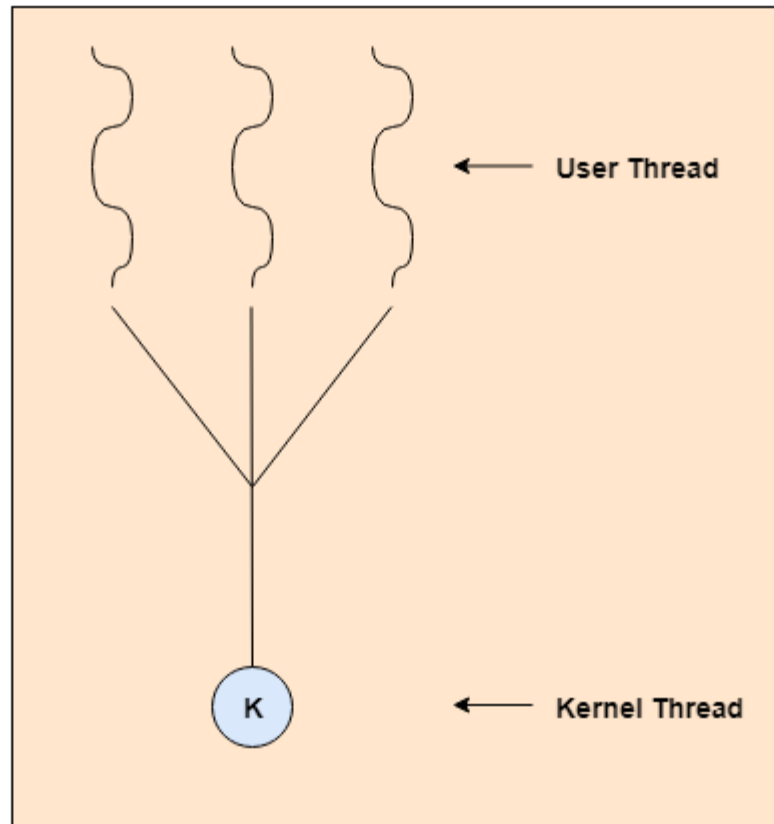
Multithreading Models – Many to One Thread

- Multiple user threads share a single kernel thread.
- User threads are scheduled by the runtime environment and Kernel thread was scheduled by the operating system.
- By using a thread library, the management of the thread can be done in the user space.
- When the blocking system call is made by a thread, the entire process of the thread gets blocked.

Multithreading Models – One to One

- Here, a single user-level thread is mapped to a single kernel-level thread.
- It provides more concurrency than the many-to-one model by allowing another thread to run, when a thread makes a blocking system call.
- It also allows multiple threads to run in parallel on multiprocessors.
- The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system.
- The Windows and Linux operating systems aim to solve this problem by restricting the number of threads created.

Multithreading Models – One to One



Many to One Model

Multithreading Models – Many to Many

- The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.
- The number of kernel threads may be specific to either a particular application or a particular machine.
- If a thread performs a blocked system call under this model, the kernel can schedule another thread to execute.
- This model provides the best accuracy and concurrency on a thread.

Multithreading Models – Many to Many

